EXPERIMENT NO: 5

Student Name:Divyansh **UID:** 23BCS11778

Branch: BE-CSE **Section/Group:** KRG-2A

Semester: 6th **Date of Performance:** 25/02/2026

Subject Name: System Design **Subject Code:** 23CSH-314

Aim

To design a scalable real-time messenger application similar to WhatsApp or Facebook Messenger that allows users to register, send and receive messages, create groups, share media, and maintain real-time communication by identifying functional and non-functional requirements and defining system APIs.

Objectives

- Understand the architecture of a large-scale real-time messaging application.
- Identify functional requirements such as user registration, messaging, and group chats.
- Identify non-functional requirements including scalability, availability, and low latency.
- Analyze CAP theorem trade-offs in messaging systems.
- Design RESTful APIs for user management and message handling.Identify functional requirements such as user onboarding, subscriptions, search, and video play- back.

Procedure

- Studied real-world messaging platforms such as WhatsApp and Facebook Messenger.
- Identified core entities including User, Message, Chat, Group, and Media.
- Analyzed user flow from registration to real-time message delivery.
- Collected functional and non-functional requirements.
- Designed APIs for user onboarding, messaging, and group management.
- Analyzed scalability requirements for millions of concurrent users.
- Studied availability and consistency trade-offs for message delivery and read receipts.

Functional Requirements

1. Users should be able to create an account and log in securely.
2. Users should be able to send and receive real-time text messages.
3. Users should be able to create one-to-one and group chats.
4. Users should be able to send media files such as images, videos, and documents.
5. Users should be able to see message delivery and read status (Optional).

Core Entities of the System

• User
• Message
• Chat
• Group
• Media
• Notification Service

API Design

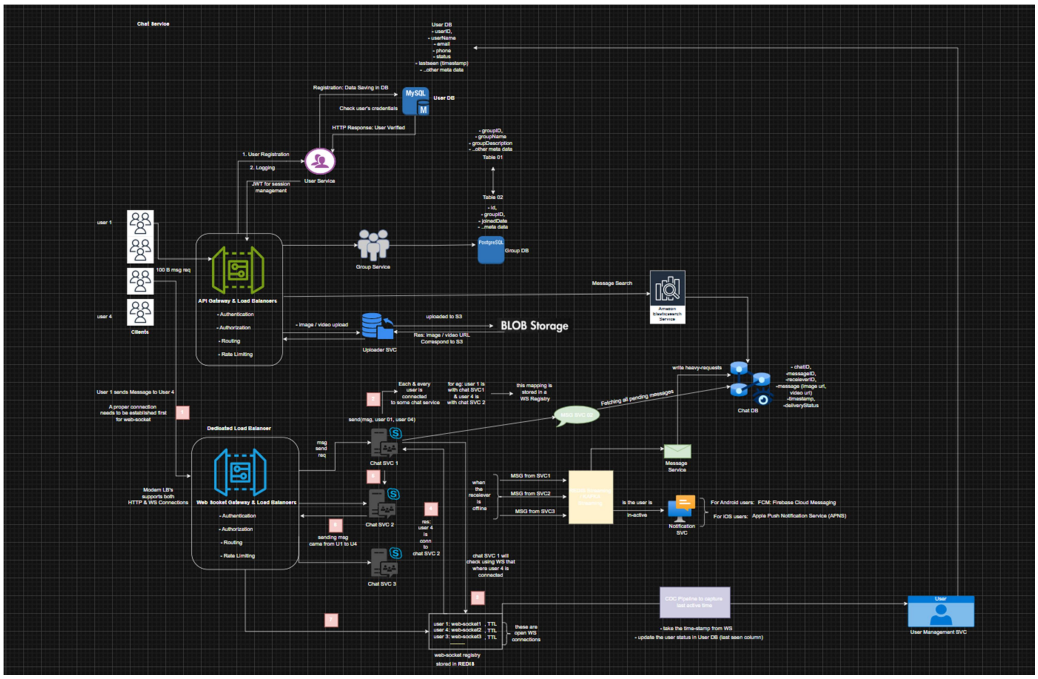
1. User Registration API: **POST /user/register**
2. User Login API: **POST /user/login**
3. Update User Profile API: **PUT /user/update**
4. Create Chat API: **POST /chat/create**
5. Send Message API: **POST /message/send**
6. Fetch Messages API: **GET /messages/{chat_id}**
7. Create Group API: **POST /group/create**
8. Send Media API: **POST /media/upload**

Non-Functional Requirements

1. Scalability to support hundreds of millions of active users.
2. High availability with minimal downtime.
3. Low latency message delivery (less than 100 ms).
4. Eventual consistency for message delivery and strong consistency for user authentication.
5. Horizontally scalable architecture using distributed databases and message queues.

High Level Design (HLD)

The system consists of Client Applications, API Gateway, Authentication Service, Messaging Service, Group Service, Media Service, Notification Service, Cache, Message Queue, and Distributed Databases. Messages are delivered in real-time using WebSockets, while media files are stored in object storage and delivered through CDN for efficient access



Outcome

- Designed a scalable real-time messaging application.
- Identified functional and non-functional requirements.
- Designed RESTful APIs for messaging, user management, and group chats.
- Understood scalability challenges and CAP theorem trade-offs in messaging systems.